Infrastructure as Code

Configuration Management

Ron C. Chiang

University of St. Thomas



Agenda

- Application Deployment Strategies
- Immutable Infrastructure
- Packer
- Configuration Management
- Ansible

Deployment strategies

- Question: Once an application is built, how do we deploy it to distributed infrastructure?
  - Strategy for deploying to 1 server very different than 100's.
  - Trade-offs between deployment speed and risk.
  - Cloud elasticity and pricing model supports new strategies.

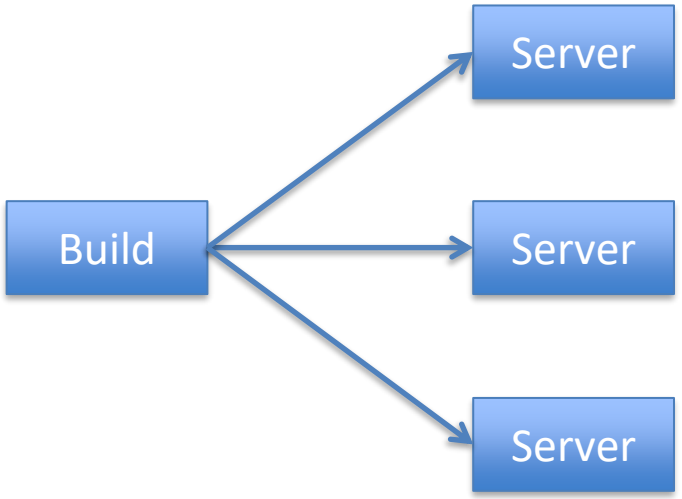
Deployment strategies

- **Single Server Deployment**
  - Application is deployed to a single server.
  - Very common for testing and small production applications.
  - Benefits:
    - Easy and quick to deploy app on a single server.
  - Problems:
    - Service outage occurs during app deployment
    - No ability to test app in production before deployment
    - Rolling back app requires complete reinstallation



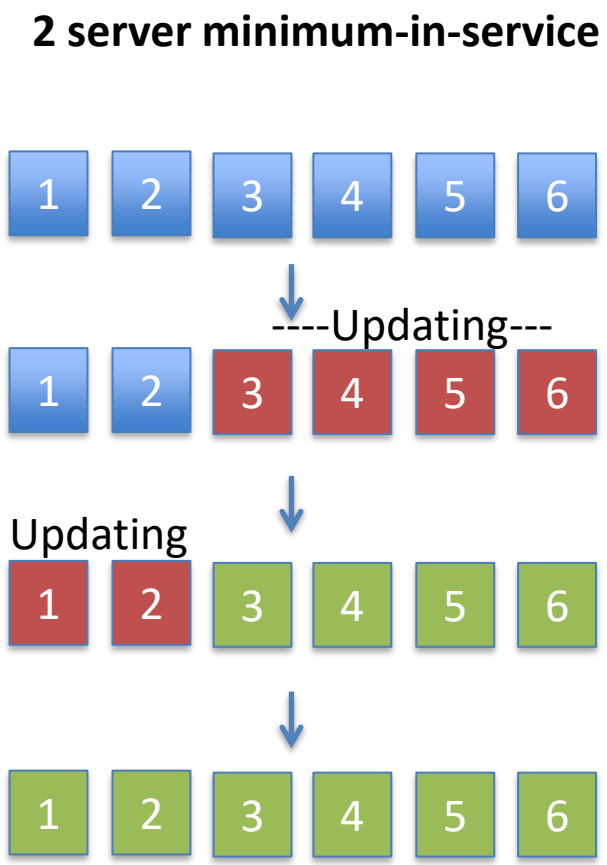
Deployment strategies

- **Big Bang Deployment**
  - Application is deployed to all servers at once.
  - Common for a small set of servers (2-5)
  - Benefits:
    - Quick to deploy app on multiple servers, though requires orchestration.
  - Problems:
    - Service outage occurs during app deployment.
    - No ability to test app in production before deployment.
    - Rolling back app requires complete reinstallation.



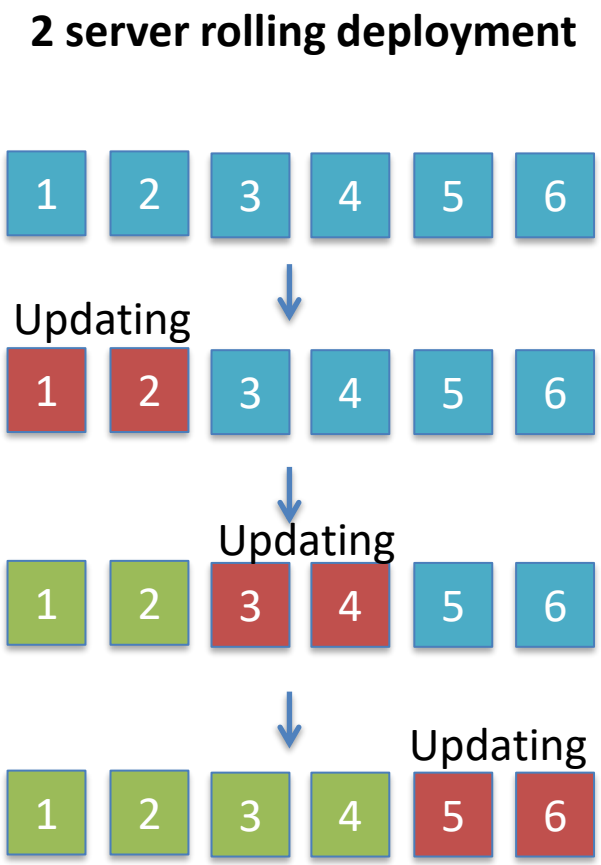
Deployment strategies

- **Minimum-in-service Deployment**
  - Application is deployed in multiple stages with a minimum set of servers supporting service.
  - Benefits:
    - Supports large environments
    - No downtime
    - Fast, most systems updated
    - Deployments are tested in production
  - Problems:
    - Requires significant automation and orchestration capabilities
    - May require rollback



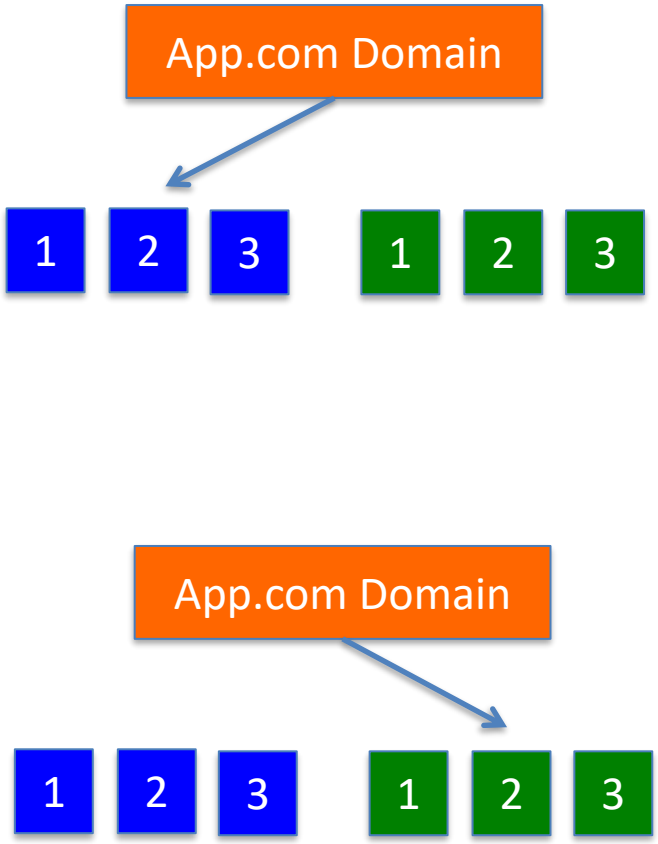
Deployment strategies

- **Rolling (Canary) Deployment**
  - Application is deployed to a specified number of servers at each stage
  - Benefits:
    - Low risk strategy
    - Maintains health of service
    - Minimize user impact through early service issue detection
  - Problems:
    - Requires significant automation and orchestration capability
    - Slow, depending on number of servers updated at each stage



Deployment strategies

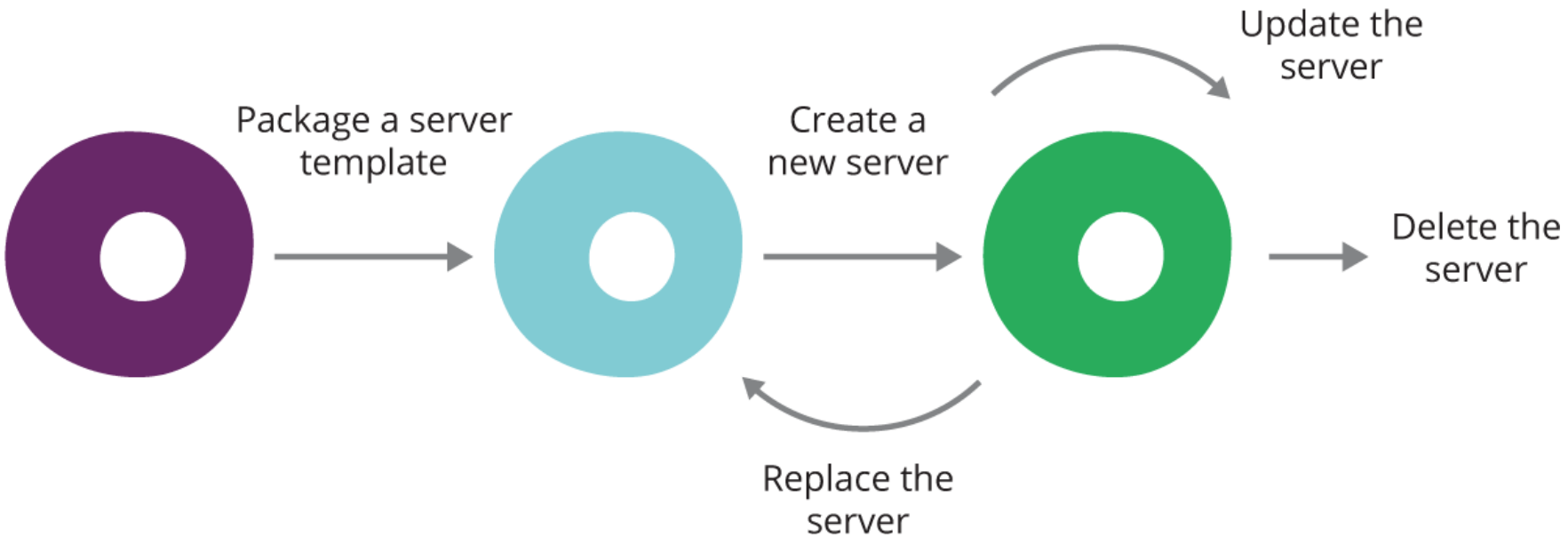
- **Blue Green Deployment**
  - Application is deployed to a completely new production environment in parallel.
  - Benefits:
    - Maintains health of service
    - Rapid cut-over deployment
    - Ability to test complete deployment in production
    - Easy to perform full rollback
  - Problems:
    - Requires significant automation and orchestration capability
    - Extra cost, but manageable with cloud instances



Deployment Challenges Summary

- Minimizing disruption to customer services.
- Supporting multiple versions of application simultaneously.
  - Database scheme must support multiple app versions.
  - Application APIs must support multiple contract versions.
- Gracefully recovering from failure.
  - Rolling back updates -- what about the database?
  - Maybe we just roll forward…

Application Server Lifecycle



Create a new server

- How do we create new servers using automation?
  - Server consists of:
    - Infrastructure definition (server type)
    - Operating system
    - Software (php, apache, mysql)
    - OS & application configuration
    - Data
  - Use Terraform, CloudFormation, Auto-scaling or other tools to create the infrastructure.
  - Select a server template (image) containing an operating system and software.
  - Deploy software and configuration using a command tool (Capistrano) or configuration management tool (Chef/ Puppet/ Ansible).
  - Potentially package everything in containers (Docker – Lecture 12).

Server Template

- There are two general approaches to building a server:
  - **Bake**: pre-generate a software image which contains an OS and all the required software components.
    - Example: Use an AMI based on a snapshot of a fully working system.
    - May need to perform a few configuration tasks once it's launched.
    - Benefit: Allows you to launch a new server quickly.
    - Risk: May require you to frequently create and manage new images.
    - Practice: minimize amount of software in a template.
  - **Fry**: launch a server with a plain OS distribution and then install required software on-the-fly.
    - Example: Use a base AMI image and run a userdata shell script.
    - Benefit: No need to manage images.
    - Risk: New instances may take a long time to deploy.

Packer

- Packer is a tool to create multiple machine images based on a configuration file.
  - Can create images for AWS, VMWare, Docker, and others.
  - Configuration file uses JSON.
  - Developed by HashiCorp (Terraform & Vagrant).
  - Free tool available at: http://www.packer.io

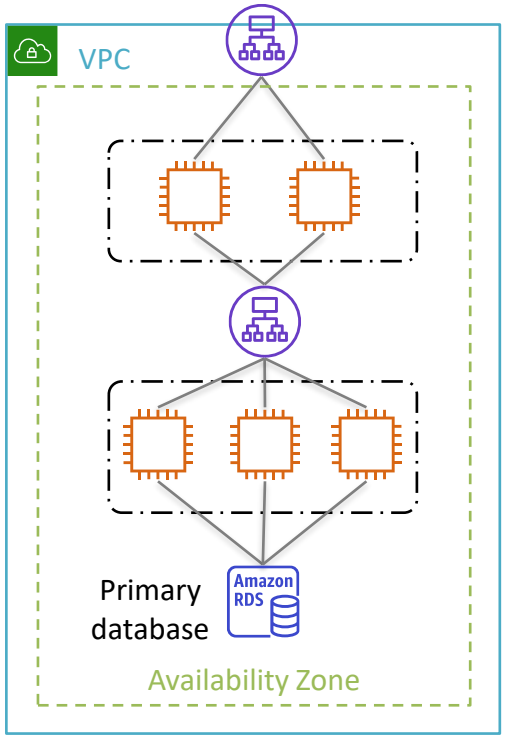
- The configuration file defines:
  - **Builders**: describe the type of machine image to create and what infrastructure is used to create it.

  - **Provisioners**: describe what software components to install on the machine image.

How will you keep your fleet updated?



You might have hundreds of instances to manage.
How do you apply guest operating system (OS) patches and update the software that is installed on them?



Update a server

- 4 models for making configuration changes to servers:
  - **Ad hoc** change management: leave server alone until a specific change needs to be made and manually make the change.
    - Tends to leave servers inconsistently configured.
    - Leads to **configuration drift** – a set of servers identically configured will gradually drift apart.
  - **Continuous configuration synchronization**: server configuration tool runs on an unattended schedule and applies the current set of configuration definitions.
    - Minimizes configuration drift by continuously reapplying configuration.
    - Only manages configuration covered by definition files, creating opportunities for other parts of the server configuration to drift.

Update a server

- **Immutable servers**: a deployment methodology where we replace systems instead of updating them.
  - Allows us to fully test configuration changes and not worry about unexpected effects of changing running systems.
  - Eliminates pets from the infrastructure environment.
  - Mitigates configuration drift because systems don't live long.
  - Practice: replace servers anytime a server template changes.
  - **Phoenix server** pattern: replace servers on a scheduled basis regardless of whether changes need to be made or not.
- **Containerized servers**: use containers to package applications and their dependencies.
  - Basically immutable servers, but with containers.

Dynamic Infrastructure Challenges

- Server sprawl
  - It's really easy to provision new servers.
  - It becomes difficult to patch and consistently maintain servers and the numbers grow.
- Configuration drift
  - Configuration of servers gradually change over time.
  - Unmanaged variation leads to support and maintenance challenges.
- Snowflake servers
  - Unique servers that cannot be easily replicated.
  - Leads to fragile infrastructure and components that nobody knows how to fix.
- Automation fear
  - Fear of running automation tools because so much infrastructure is hand-cranked.
- Erosion
  - Natural entropy suggests that problems will creep into systems over time.

Continuous synchronization

- Regularly run a process to apply the current configuration definitions to a server.

- Three synchronization patterns:
  - **Pushing to synchronize**: a master server pushes configuration to the servers it manages.
    - Typically uses common communication channels like SSH.
    - No need to install special software agent on managed servers.
    - Centralized control over when changes are distributed to servers.
  - **Pulling to synchronize**: each managed server pulls configuration from a master server.
    - Polling is controlled by an agent on each managed server.
    - May be more secure because an incoming communications channel does not need to be always open on the server.
    - May be more scalable than a push-based synchronization architecture.
  - **Masterless configuration management**: each managed server downloads copies of the configuration definitions and runs a configuration management tool in a local mode.
    - Eliminates a central server as a single point of failure.

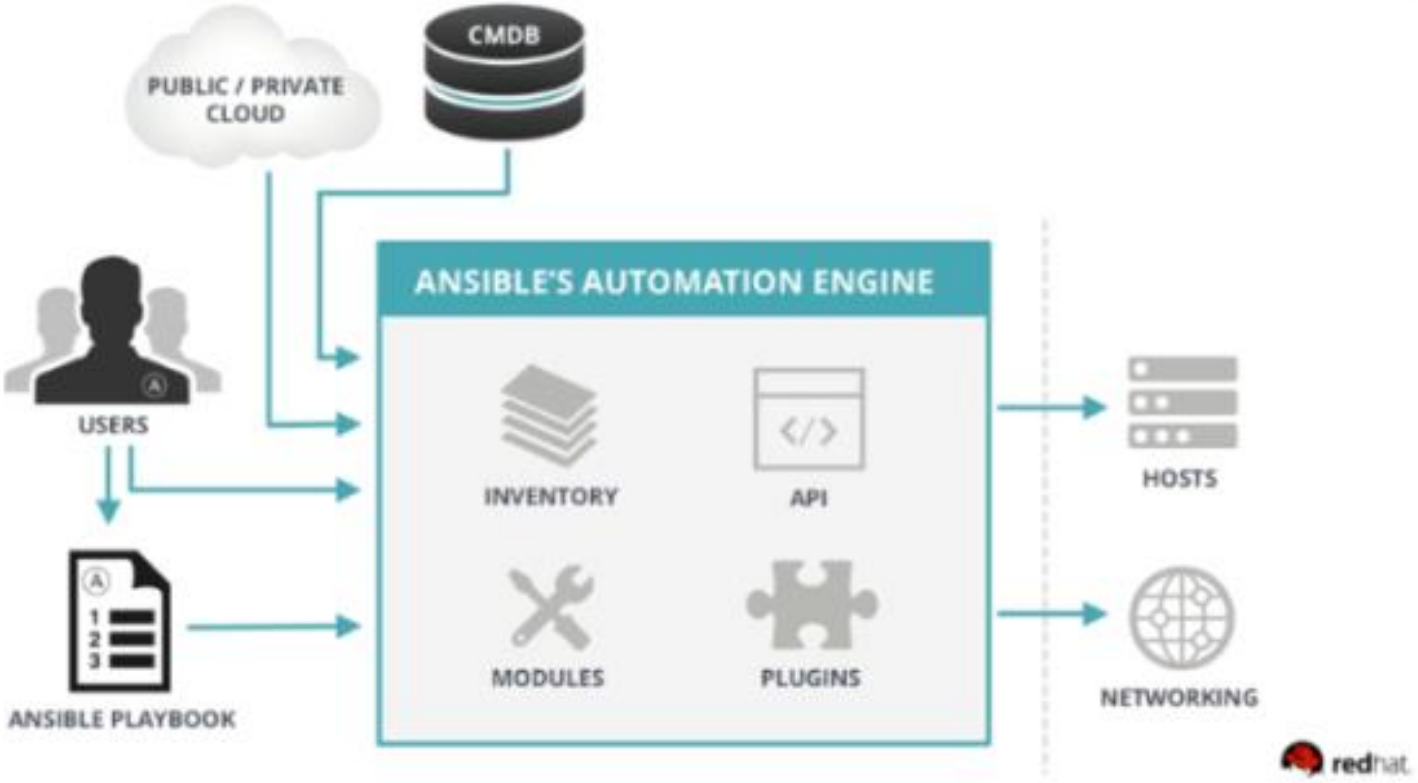
Ansible



- A free-software platform for configuring and managing servers.
  - Push-based configuration using SSH as a communications channel – no agents!
  - Supports both synchronized and ad hoc task execution on Linux and Windows systems.
  - Uses YAML to describe the configuration of systems.
  - Python over SSH.
  - Originally developed by Michael DeHaan and acquired by Red Hat.
  - Written in Python and available at www.ansible.com
    - Control application must run on Unix/Linux system

ANSIBLE ARCHITECTURE

ANSIBLE



Playbooks

- **Playbooks** are Ansible's configuration, deployment, and orchestration language.

- Plain-text YAML files that describe the configuration state of a system.

Playbooks contain **plays**.
Plays contain **tasks**.
Tasks call **modules**.

- Tasks run sequentially.

- **Handlers** may be triggered by tasks, and run once at the end of plays.

Example playbook

```
---
- name: install and start apache
  hosts: web
  remote_user: justin
  become_method: sudo
  become_user: root
  vars:
   http_port: 80
   max_clients: 200

  tasks:
  - name: install httpd
    yum: name=httpd state=latest
  - name: write apache config file
    template: src=srv/httpd.j2 dest=/etc/httpd.conf
    notify:
    - restart apache
  - name: start httpd
    service: name=httpd state=running

  handlers:
  - name: restart apache
    service: name=httpd state=restarted
```

Inventory

- Ansible uses an inventory file, named **`hosts`**, to store a list of systems that may be configured.
  - The hosts file uses an INI-like format:

```
[webservers]
web1 ansible_host=54.162.154.245 ansible_user=ec2-user
web2.mydomain.com ansible_user=ec2-user

[databases]
db[01:10].mydomain.com maxRequests=1000
```

  - Hosts may be members of one or more groups.

Inventory Variables

- Variables hold values that hosts or groups of hosts can use for configuration.
  - Dynamically alter how playbooks run.

- We can pass in variables via the inventory (**group_vars** or **host_vars** directories), command line, or using facts.

- Example for ../group_vars/webservers.yml

```
---
ntp_server: ntp.google.com
database_server: storage.mydomain.com
```

Configuration file

- Ansible uses a configuration file to control specific software settings.
  - Stored in an INI format.
  - User can override default settings.

- Where Ansible looks for configuration settings:

  - ANSIBLE_CONFIG **(**an environment variable**)**
  - ansible.cfg **(**in the current directory)
  - .ansible.cfg **(**in the home directory**)**
  - /etc/ansible/ansible.cfg

- A convenient configuration setting is to define the location of the inventory file:

```
# ansible.cfg file
[defaults]
inventory = /home/username/configuration/hosts
```

Host Selection Patterns

- We use patterns to decide which groups of hosts to manage.

all	(all hosts, can also use *)
*.com	(all hosts ending in .com domain)
web1.mydomain.com	(single host)
webservers:dbservers	(webservers OR dbservers)
webservers:!web1	(webservers, except for web1)
webservers:&mpls	(intersection of webservers & mpls)
webservers[1]	(2nd host listed in webservers group)

Modules

- Modules are helper applications that Ansible uses to automate every part of your server configuration.
  - Over 450 modules available and you can create your own.
- We pass in directives to modules to control behavior.
- Standard structure:

```
module: directive1=value directive2=value
```

Role

- A role is a special kind of re-usable playbook that is fully self-contained:
  - Tasks
  - Variables
  - Configuration templates
  - Supporting files
- We can use a role to define how to build a specific type of system, like a webserver.
- Ansible Galaxy: global repository where Ansible users share roles.
  - https://galaxy.ansible.com

Running Ansible

- 3 ways to run Ansible:
  - **Ad-hoc**: ansible <inventory> -m <module>
  - **Playbooks**: ansible-playbook <YAML file>
  - **Automation Framework**: Ansible Tower

Ad hoc commands

- Ansible can be used to execute quick commands on hosts.
  - Used for very small tasks or testing.

```
$ ansible <host pattern> -m <module_name> -a <arguments>

$ ansible web1 -m command –a “uptime”
```

- The **command** module is the default module, so we don’t have to specify it.

```
$ ansible all –a “tail /var/log/yum.log” –-become
```

- Ansible will execute the command as the user specified by the –u option.

Ad hoc commands

- Copy files to a group of servers using **copy** module:

```
$ ansible minneapolis –m copy –a “src=~/archive.zip dest=/var/www/archive.zip”
```

- Create and delete a directory using **file** module:

```
$ ansible webservers -m file –a “dest=/path/to/files mode=755 state=directory”
$ ansible webservers –m file –a “dest=/path/to/files state=absent”
```

- Install a software package using **yum** module:

```
$ ansible dbserver –m yum –a “name=mysql state=present”
```

Ad hoc commands

- Create and remove user accounts with **user** module:

  $ ansible web1 -m user –a “name=jason password=<crypted password>”
  $ ansible web1 -m user –a “name=jason state=absent”

- Manage services using **service** module:

  $ ansible webservers -m service –a “name=httpd state=started”
  $ ansible webservers –m service –a “name=httpd state=restarted”

- Facts represent a set of values which describe the configuration of a system. Gather facts using the **setup** module:

  $ ansible web1 –m setup

Ansible Hands-on



- Launch Ansible hands-on environment:
  - https://seis665-public.s3.amazonaws.com/AnsibleSystems.json
- Run ad-hoc commands (within ansible directory):
  - Ping command:

```
$ ansible web1 –m ping
```

  - Uptime:

```
$ ansible web1 -m command –a “uptime”
$ ansible web1 –a “uptime”
```

  - Top (sorted by memory allocation):

```
$ ansible web1 –a “top -abn 1”
```

Ansible Hands-on



- Install nginx package:

```
$ ansible web1 –m yum --become -a “name=nginx
state=present”
```

- Re-run nginx package install.
  - Commands are idempotent.
  - Manual changes to system will be removed to correct configuration drift.

- Remove nginx package:

```
$ ansible web1 –m yum --become -a “name=nginx state=absent”
```

Check Mode

- Ansible can perform a dry-run for ad-hoc commands and playbooks.
  - Implemented with the –C flag on command line.

- Used to validate Ansible execution before making state changes on target systems.
  - Note: Not all modules support check mode.

```
$ ansible web1 –C –m –yum –a “name=httpd state=latest”
$ ansible-playbook –C my-playbook.yml
```

Playbooks

- Let's start with something easy:

```
$ ansible all –m shell –a 'echo hello world'
```

- We can turn this into a playbook called helloworld.yml containing a single play:

```
---
-   hosts: all
     tasks:
          - shell: echo "hello world"
```

```
$ ansible-playbook -v helloworld.yml
```

- Execute a playbook against multiple systems:

```
$ ansible-playbook <playbook> -f <parallelism level>
```

```
$ ansible-playbook playbook.yml -f 10
```

Playbooks

- Playbooks can declare configurations, but also orchestrate steps of complex ordered processes.
  - Bounce back-and-forth between sets of machines.
  - Launch tasks synchronously or asynchronously.

- Playbooks contain one or more plays in a list.
  - A play maps a group of hosts to well defined roles, represented by tasks.

Plays

- We can define the hosts to target and remote user to perform tasks for each play:

```
---
- hosts: webservers
  remote_user: ec2-user
  tasks:
      - name: test connection
        ping:
```

- Some tasks require root access to execute properly and we can **become** a super-user:

```
---
- hosts: webservers
  remote_user: ec2-user
  become: yes
  become_method: sudo
  tasks:
      - yum: name=nginx state=present
      - service: name=nginx enabled=yes
```

Tasks

- Each play contains a list of tasks which are executed in order, one at a time.
  - Tasks are executed against matched host pattern.
  - All hosts receive the same task directives.
  - Hosts with failed tasks are removed from the entire playbook.
- Each tasks executes a module with specified arguments.
  - Variables may be used as arguments for modules.
- Modules are idempotent and will only make the changes necessary to bring the system to the desired state.
- Every task should have a **name** which is included in the output from running the playbook.

Tasks

- A task specifies a module and a set of key/value arguments:

```
tasks:
  - name: make sure apache is running
    service: name=httpd state=started
```

- The command and shell modules just take a list of arguments vs. key/value:

```
tasks:
  - name: disable selinux
    command: /sbin/setenforce 0
```

- Tasks can also use variables as arguments:

```
tasks:
  - name: create a virtual host file for {{ vhost }}
    template: src=somefile.j2 dest=/etc/httpd/conf.d/{{ vhost }}
```

Includes

- We can use include files to reuse tasks between plays or playbooks.

```
tasks:
  - include: tasks/foo.yml
```

- We can also pass variables into includes:

```
tasks:
  - include: wordpress.yml wp_user=timmy
  - include: wordpress.yml wp_user=alice
  - include: wordpress.yml wp_user=bob
```

- It’s also possible to include one playbook file into another:

```
- name: this is a play at the top level of a file
  hosts: all
  tasks:
    - name: say hi
      shell: echo "hi..."

- include: load_balancers.yml
- include: webservers.yml
```

Variables

- Variables allow us to deal with small configuration differences between systems.
- Variable names should be letters, numbers, and underscores.
  - Should always start with a letter.
- YAML can map keys to values:

```
foo:
  field1: one
  field2: two
```

- Multiple ways to reference a specific field in a dictionary value:

```
foo['field1']
foo.field1
```

- Define variables directly in a playbook using **vars** section:

```
- hosts: webservers
  vars:
    http_port: 800
```

Facts

- Facts are a type of variable that contain information about remote systems.
  - System name, IP address, OS version, etc
  - Facts are discovered automatically by the **setup** module and stored in a JSON structure.

```
"ansible_architecture": "x86_64",
"ansible_bios_date": "09/20/2012", "ansible_bios_version": "6.00",
"ansible_swapfree_mb": 665,
"ansible_swaptotal_mb": 1021,
"ansible_system": "Linux",
"ansible_system_vendor": "VMware, Inc.",
"ansible_user_id": "root”
…
```

- We can refer to a fact in a play like any other variable:
  {{ ansible_architecture }}

$ ansible all -m setup

Registered Variables

- Registered variables contain the output returned by executing a command.

```
- hosts: web_servers

  tasks:

     - shell: /usr/bin/foo
       register: foo_result
       ignore_errors: True
```

- The variable foo_result will contain the output from running the /usr/bin/foo command on a remote host.

Templates

- **Templates** allow us to dynamically create configuration files using variable substitution.
  - Uses the Jinja2 template format.

```
# nginx.conf.j2 template
server {
    listen       {{ webserver_name }}:8080;
    server_name  {{ webserver_name }}
    alias  {{ website_alias }};

    location / {
        root   html;
        index  index.html index.htm;
    }
}
```

- Copy a template to a remote host:

```
template: src=nginx.conf.j2 dest=/etc/nginx/nginx.conf
```

Jinja2 Filters

- Jinja2 supports a number of filters we can use in templates to manipulate variables.
  - Set a default value for a variable:
    {{ some_variable | default(5) }}
  - The maximum value from a list of numbers:
    {{ [3, 4, 2] | max }}
  - Get a random item from 0 to 10:
    {{ 10| random }}
  - Test if a string is a valid IP address:
    {{ myvar | ipaddr }}
  - Concatenate a list into a string:
    {{ var_list | join(“ ”) }}

Conditionals

- Use the **when** statement to skip a particular task from executing on a host.

```
tasks:
  - name: "shut down Debian flavored systems"
    command: /sbin/shutdown -t now
    when: ansible_os_family == "Debian”
```

- Note that Ansible facts and vars like ansible_os_family can be used directly in conditionals without double curly braces.

- We can specify that multiple conditions need to be true:

```
tasks:
  - name: "shut down CentOS 6 systems"
    command: /sbin/shutdown -t now
    when:
      - ansible_distribution == "CentOS"
      - ansible_distribution_major_version == "6"
```

Loops

- Oftentimes it's useful to repeat tasks to save some typing.
  - Loop using **with_items** directive.

```
- name: add several users
  user: name={{ item }} state=present groups=developers
  with_items:
    - jadams
    - msmith
    - cmilwaui
```

- Items in loops do not have to be simple strings:

```
- name: add several users
  user: name={{ item.name }} state=present groups={{ item.groups }}
  with_items:
    - { name: 'testuser1', groups: 'wheel' }
    - { name: 'testuser2', groups: 'root' }
```

Loops

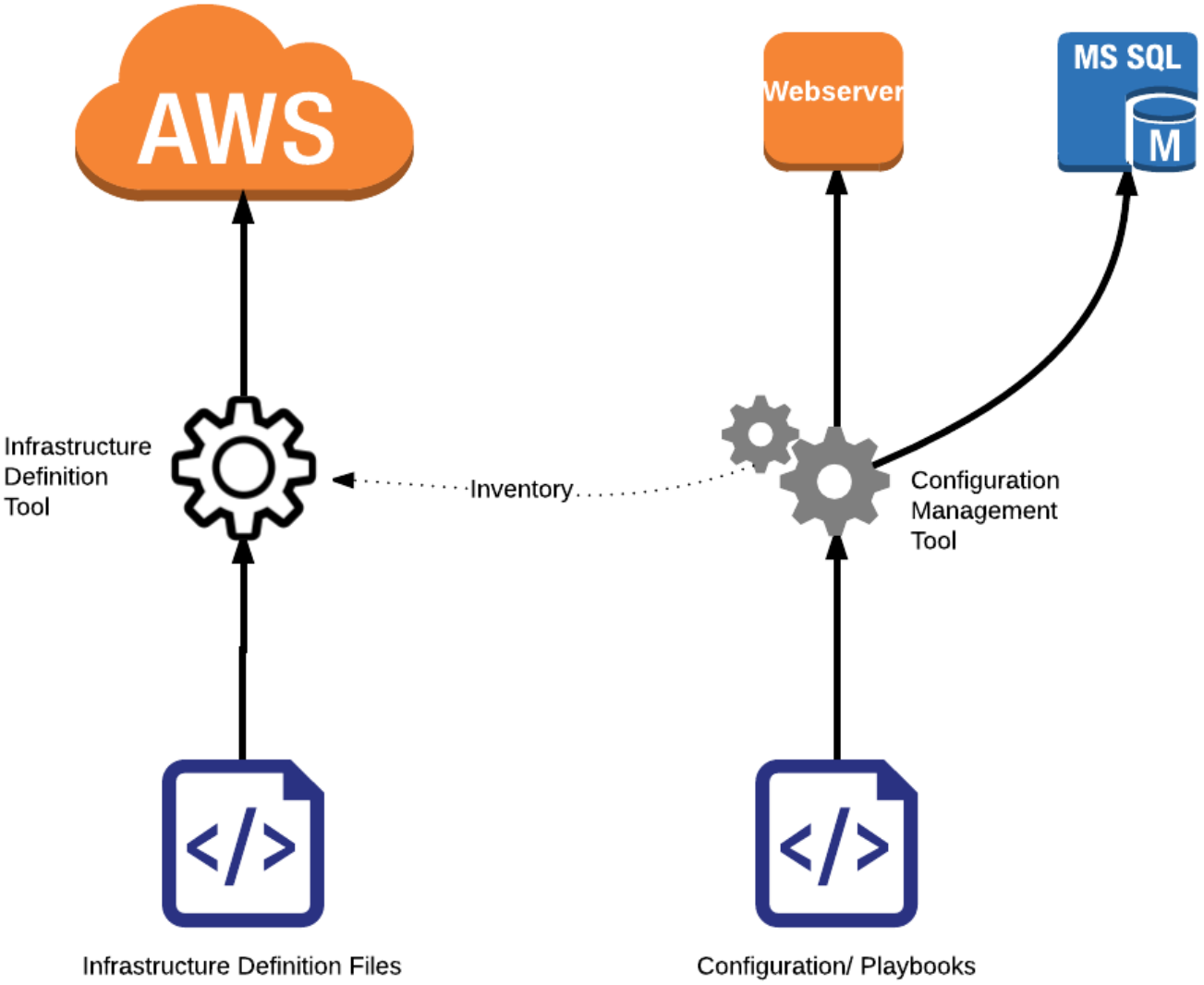
- The with_sequence directive generates a sequence of items in ascending order.

```
# create a series of directories with even numbers for some reason
    - file: dest=/var/stuff/{{ item }} state=directory
      with_sequence: start=4 end=16 stride=2
```

- Sometimes we want to retry a task until a certain condition is met.

```
- action: shell /usr/bin/foo
  register: result
  until: result.stdout.find("all systems go") != -1
  retries: 5
  delay: 10
```

Putting it all together



Resources

- Ansible documentation:
  - http://docs.ansible.com/
- Ansible module index:
  - https://docs.ansible.com/projects/ansible/latest/collections/index_module.html
- Ansible Absolute Basic Overview video:
  - https://www.youtube.com/watch?v=MfoAb50Br94

AWS Systems Manager



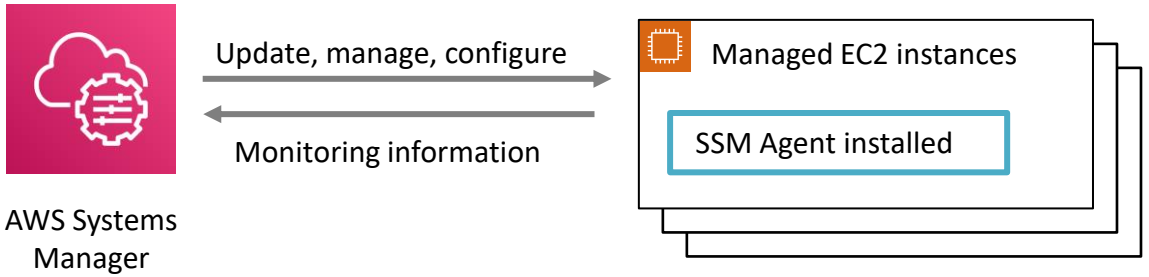
AWS Systems Manager

Gain operational insights and take action on AWS resources.

- Automates operational tasks
  - Example: Apply OS patches and software upgrades across a fleet of EC2 instances
- Simplifies resource and application management
  - Manage software inventory
  - View detailed system configurations across the fleet
- Manages servers on-premises and in the cloud

© 2020 Amazon Web Services, Inc. or its Affiliates. All rights reserved.

Systems Manager capabilities



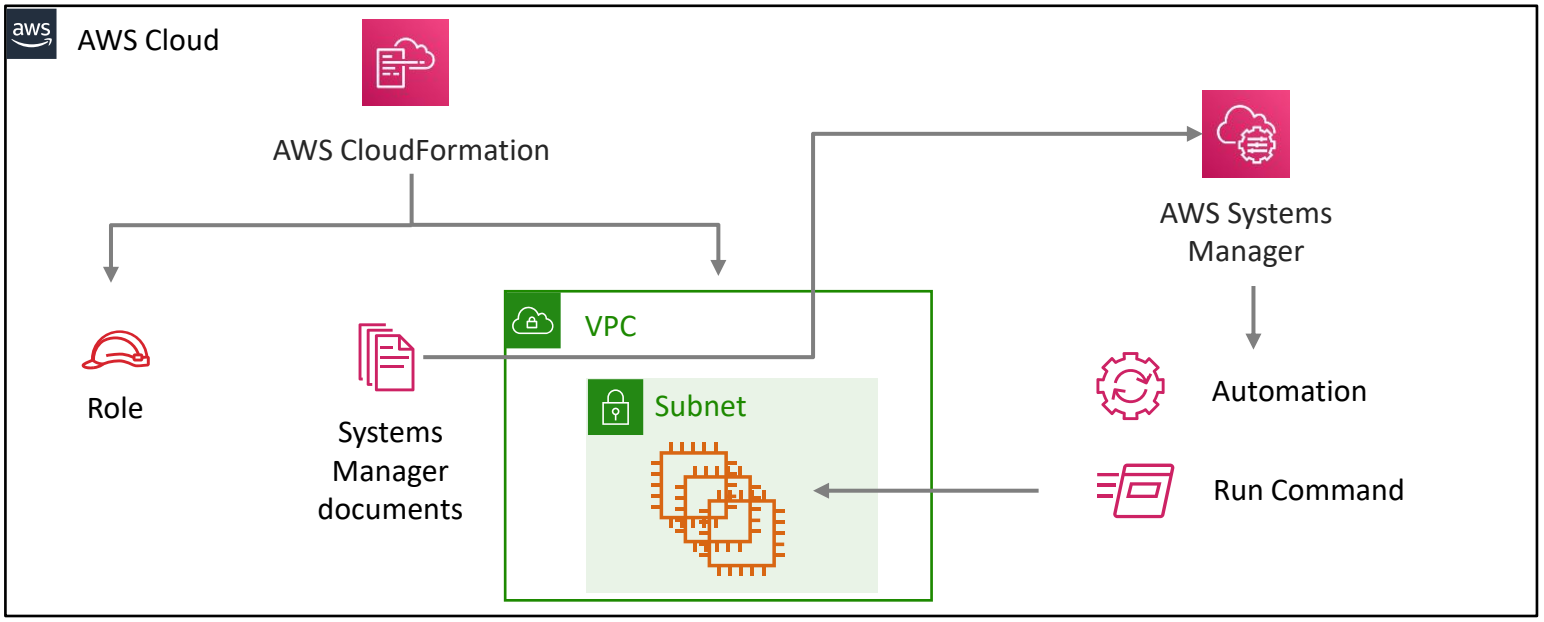
- Run Command
- Patch Manager
- Session Manager
- Maintenance Windows
- State Manager
- Inventory
- Parameter Store
- Automation
- Documents

© 2020 Amazon Web Services, Inc. or its Affiliates. All rights reserved.

AWS CloudFormation and Systems Manager complement each other

AWS CloudFormation works well for defining AWS Cloud resources.

Systems Manager works well for automating within guest operating systems.



© 2020 Amazon Web Services, Inc. or its Affiliates. All rights reserved.